



Dashboard Interactivo de Fútbol

Visualización de datos en tiempo real con **Python** y **Streamlit**

SISTEMAS DE BIG DATA · UD5



Descripción del Proyecto

El objetivo es construir un **dashboard interactivo y funcional** que consuma una API pública de fútbol en tiempo real. Todos los datos son **reales y se actualizan automáticamente** cada 5 minutos sin intervención del usuario.

Python

Lenguaje principal y lógica de negocio

Streamlit

Framework para la interfaz web interactiva

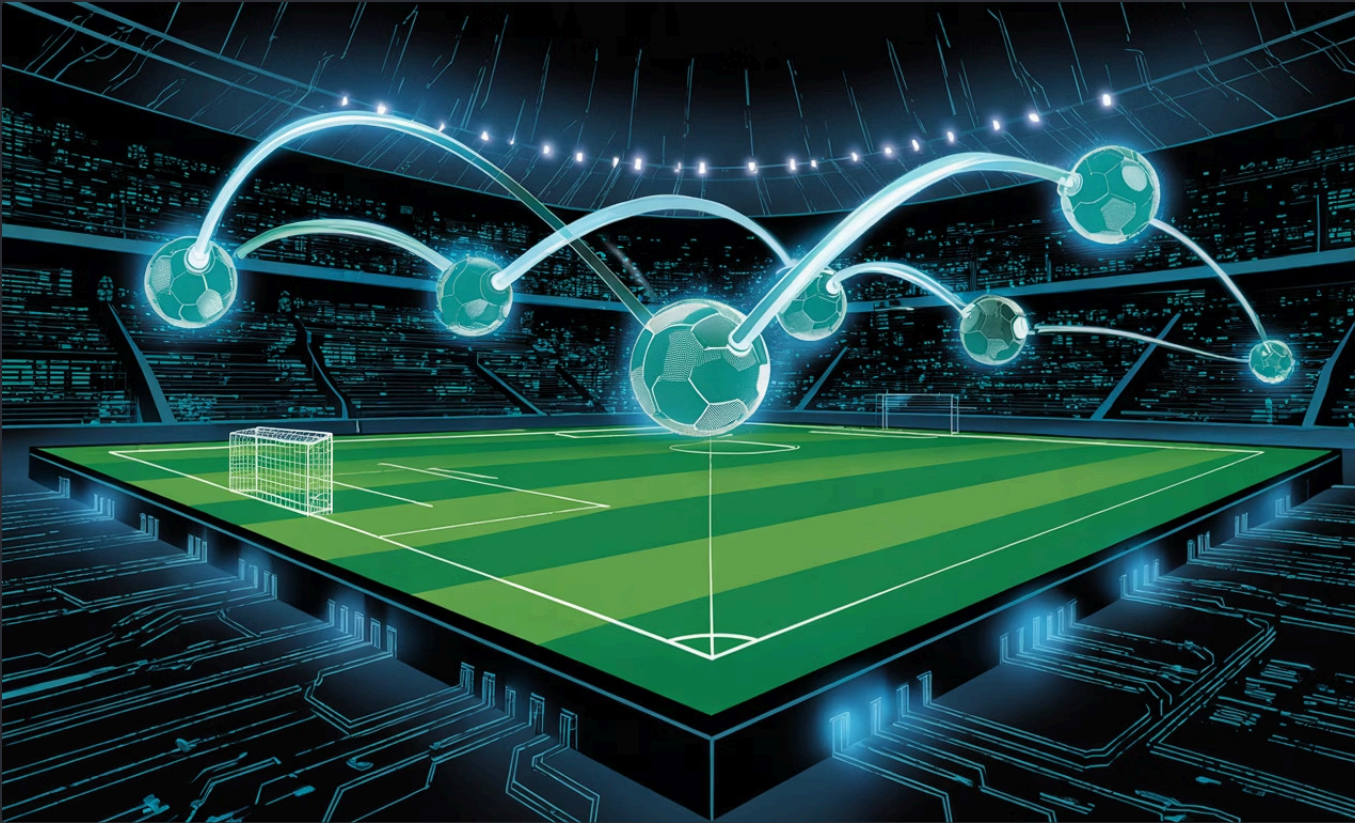
Plotly

Gráficos dinámicos e interactivos

Pandas

Manipulación y transformación de datos

Fuente de Datos



API: football-data.org

Plan gratuito con acceso a las principales ligas europeas. Los datos se cargan mediante **funciones dedicadas** — `obtener_clasificacion()` y `obtener_goleadores()` — encapsulando la lógica de consumo y garantizando un código limpio y mantenible.

 Premier League

Endpoint: `/standings`

 La Liga

Endpoint: `/scorers`

 Bundesliga

Actualización cada 5 min

 Serie A

Datos 100% reales

KPIs en Tiempo Real

Tres indicadores clave calculados dinámicamente desde los datos de la API. Se actualizan automáticamente al cambiar de liga o en cada ciclo de refresco.

1°

Líder de la Liga

Nombre del equipo y puntos acumulados
en la clasificación actual

23

Máximo Goleador

Jugador con más goles, equipo y total de
anotaciones en la temporada

2.8

Media de Goles

Total de goles en la liga dividido entre el
número de equipos

Visualizaciones con Plotly

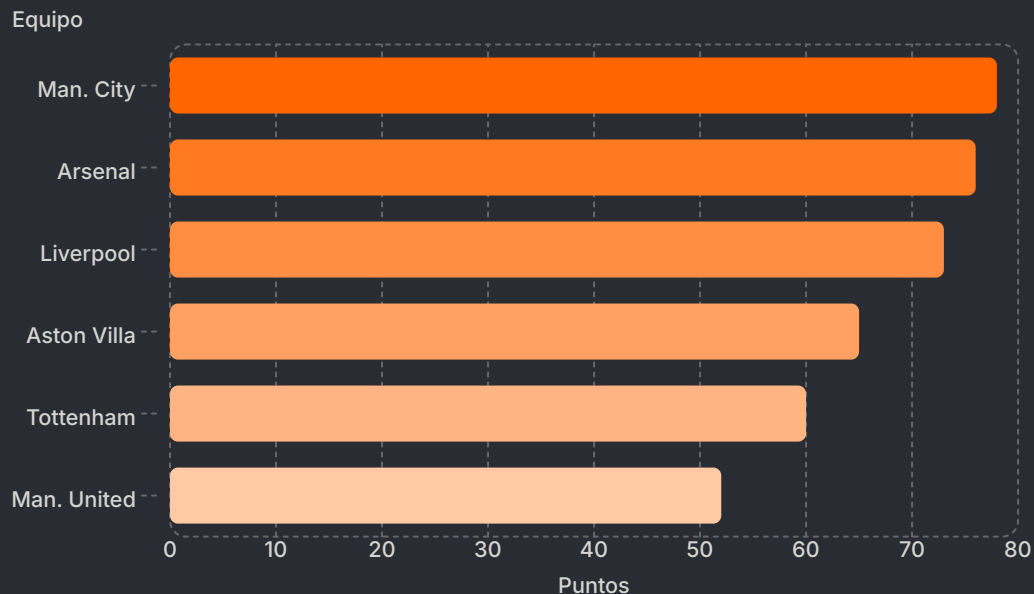


Gráfico de Barras Horizontal

Top 10 equipos ordenados por puntos. Escala de color en **teal** con intensidad proporcional a la posición.

Gráfico de Dispersión

Eje X: goles a favor · Eje Y: goles en contra. Tamaño de burbuja proporcional a los puntos. Gradiente de **rojo a verde** según el balance ofensivo-defensivo.

i Ambos gráficos se actualizan dinámicamente al seleccionar una liga diferente.

Interactividad



Control Principal: `st.radio()`

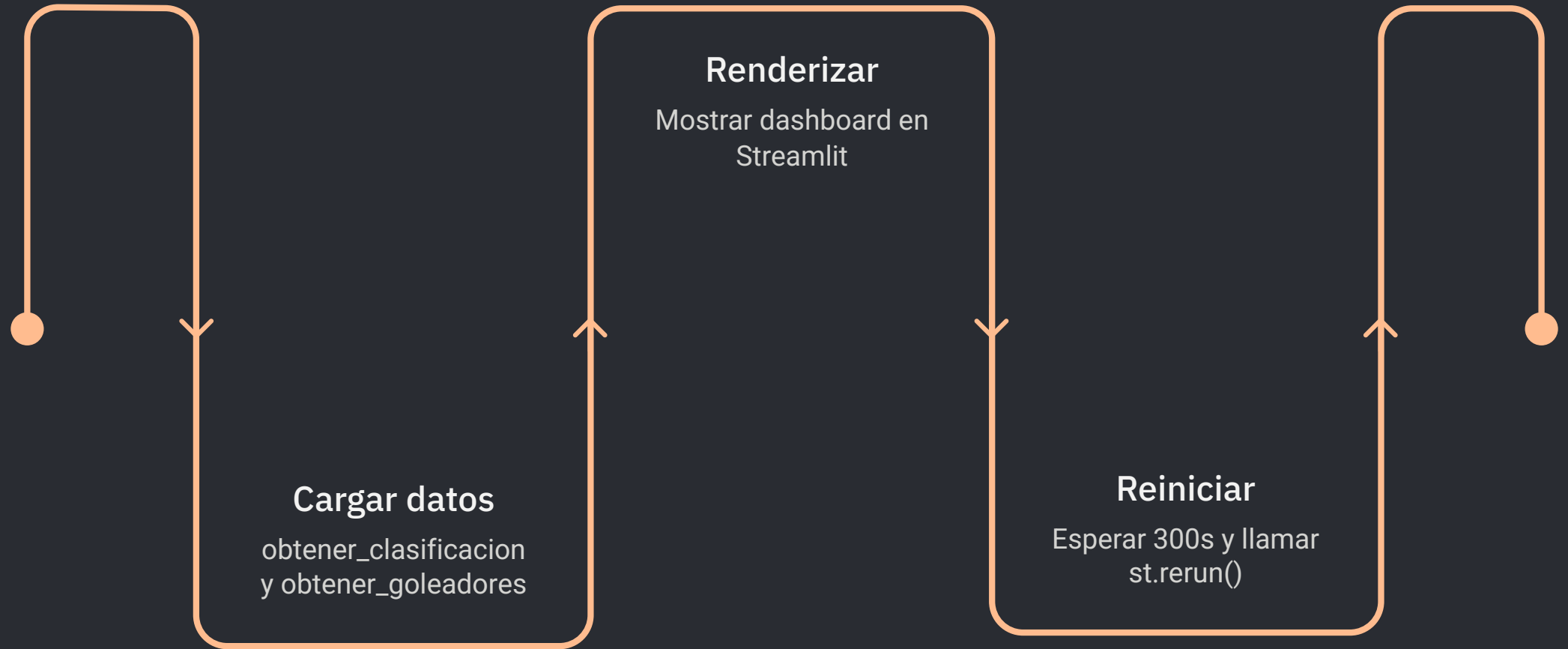
Ubicado en la barra lateral, permite seleccionar entre las cuatro ligas disponibles mediante **radio buttons** — no un dropdown genérico. Un solo clic actualiza simultáneamente:

→ Los **3 KPIs** superiores

→ El **gráfico de barras** de clasificación

→ El **gráfico de dispersión** ofensivo-defensivo

Automatización y Resiliencia



El dashboard se refresca silenciosamente cada **5 minutos** mediante `time.sleep(300) + st.rerun()`, obteniendo datos en tiempo real sin ninguna acción del usuario. Si la API no está disponible, aparece un banner de error y la aplicación **reintenta automáticamente** tras el intervalo.

Calidad del Código

```
def obtener_clasificacion(codigo_liga: str):
    """Llama al endpoint de clasificación y devuelve un
    DataFrame.
    Devuelve None si la API falla (sin internet, clave
    inválida, límite de peticiones).
    """
    try:
        url = f"
{BASE_URL}/competitions/{codigo_liga}/standings" #
Endpoint para obtener la clasificación de la liga
        r = requests.get(url, headers=HEADERS, timeout=8)
# Llamada a la API
        r.raise_for_status() # Si la
respuesta no es 200, lanza una excepción
        tabla = r.json()["standings"][0]["table"] # Tipo
TOTAL
        df = pd.DataFrame([ # Crea un
DataFrame con los datos de la clasificación
            {
                "Pos": equipo["position"], # Posición en la
tabla
                "Equipo": equipo["team"]["shortName"], #
Nombre corto del equipo
                "PJ": equipo["playedGames"], # Partidos
jugados
                "PG": equipo["won"], # Partidos ganados
                "PE": equipo["draw"], # Partidos
empatados
                "PP": equipo["lost"], # Partidos perdidos
                "GF": equipo["goalsFor"], # Goles a favor
                "GC": equipo["goalsAgainst"], # Goles en
contra
                "DG": equipo["goalDifference"], # Diferencia
de goles
                "Pts": equipo["points"], # Puntos
            }
            for equipo in tabla # Recorre la tabla de la
clasificación
        ])
        return df # Devuelve el DataFrame
    except Exception as e: # Si ocurre cualquier error
(conexión, clave, límite), lo muestra y devuelve None
        print(f"Error clasificacion: {e}")
        return None
```

Buenas Prácticas Aplicadas

Funciones modulares Cada tarea encapsulada con docstrings descriptivos	Cero duplicación Secciones organizadas con comentarios separadores
Gestión de errores Bloques try/except en cada llamada a la API	Ejecución limpia Sin errores desde la primera línea al ejecutar



Dashboard Final — Vista Completa

El resultado es una aplicación web de una sola página con arquitectura coherente: **barra lateral de control**, tres tarjetas KPI en la cabecera, visualizaciones interactivas en el centro y **tabla de clasificación completa** en la parte inferior. Todo en un fondo oscuro con acentos en teal y verde.